

Disaster Recovery Plan — Van Reservation System (VRS)

Database Platform: Amazon RDS for PostgreSQL (Target State)

Organization	Carelon Global Solutions PH
Prepared by	Arielle Jimera
Date	2026-09-18
Status	Draft v0.1 — Pending Review
Classification	Internal — Confidential

This document defines disaster recovery procedures for a single database platform variant of the Van Reservation System. A companion plan exists for the other platform variant — confirm which is in effect for the live production environment before acting on it during an incident.

1. Purpose

This document defines the Disaster Recovery (DR) Plan for the Van Reservation System (VRS), an internal Next.js/PostgreSQL application used by Cargon Global Solutions PH to book, approve, and manage company van transportation across the Manila and Iloilo sites. It replaces the manual/Nintex-based process that preceded it, so an extended outage has no automatic manual fallback.

This plan covers the database platform as targeted for the planned migration (Amazon RDS for PostgreSQL). A companion document covers the current company-managed PostgreSQL platform; the two are intentionally separate because the recovery mechanics, achievable RPO, and required infrastructure differ materially between them.

The plan exists to ensure that, in the event of a disruptive incident, the system and the people responsible for it know: what must be recovered, in what order, by whom, how success is verified, and how the incident is closed out and reported.

2. Objectives

The primary objective of this plan is to restore the Van Reservation System (VRS) to full production functionality within the Recovery Time Objective (RTO) and Recovery Point Objective (RPO) below, and to confirm — before and during any recovery — that the supporting information in this document (asset scope, contacts, recovery tasks, and the application diagram) is current and accurate.

Metric	Target	Definition
RTO (Recovery Time Objective)	4 hours	Maximum acceptable time from disaster declaration to full production functionality restored for all roles (requestor, admin) and all pages.
RPO (Recovery Point Objective)	1 hour	Maximum acceptable data loss, measured as the age of the most recent recoverable transaction relative to the moment of failure.

Asset	RTO contribution	RPO	Notes
Application (Next.js)	~45 min	N/A — stateless	Rebuilt/redeployed from source control; no state to lose.
Database (Amazon RDS for PostgreSQL)	~30–90 min	≤ 5 min (typical)	Multi-AZ automatic failover handles an AZ-level failure in roughly 60–120 seconds with no manual action. A point-in-time restore (PITR) for logical corruption or accidental data loss instead creates a new instance from continuous backups and is bounded by RDS's per-5-minute restore granularity, well inside the 1-hour RPO target.

Asset	RTO contribution	RPO	Notes
Email / notification_outbox	Resumes with app	0 — queue is durable	Queued rows survive a database failover/restore; the outbox sweeper drains the backlog after recovery.
CGS Auth API integration	Not recoverable by this team	N/A — external	Dependent on the CGS platform team's own availability; this plan cannot recover it.

Note. Amazon RDS for PostgreSQL, configured Multi-AZ with automated backups and point-in-time recovery enabled, is the platform choice that makes the 4-hour / 1-hour target comfortably and repeatably achievable without bespoke backup tooling. This is a primary driver for the planned migration away from a company-managed PostgreSQL server.

3. Scope

This plan covers the Van Reservation System (VRS) application tier, its database, its two external digital integrations, and the servers/compute that host them. It does not cover the CGS Associate Authentication API's own disaster recovery (owned by the CGS platform team), the corporate network or Active Directory that identity ultimately rests on, end-user devices, or the legacy Nintex-based process VRS replaced (deprecated, not maintained as a fallback).

3.1 Asset Scope

Applications

Asset	Description
van-reservation (web app)	Next.js 16 App Router application, TypeScript/React 19. 13 pages across three route groups: (admin) — dashboard, master-list, calendar, reports, drivers, audit-logs, roster; (auth) — login, admin/login, not-authorized; (requestor) — home, book, manage.
API routes	Route handlers under /api: auth (login/logout/session/verify-domain), associates/lookup, reservations (+ [id], + cancel), drivers, vans, admins (+ [id], + affected), audit-logs, and the internal outbox sweeper at /api/internal/dispatch-outbox.
Build/runtime	Node.js 20+, pnpm 9+. Production runs `pnpm build` then `pnpm start`. No `engines` field is pinned in package.json — version drift is a known gap, not enforced by tooling.

Databases

Asset	Description
Primary database	Amazon RDS for PostgreSQL (target engine version 17, matching the application's current dependency), Multi-AZ deployment.

Asset	Description
Schema	Identical schema to the company-managed platform: 11 tables via 13 ordered Kysely migrations (see companion PostgreSQL plan §3.1 for the full table list). Migration mechanics do not change with the platform.
Backups	RDS automated backups with point-in-time recovery (PITR), retention period to be set per company policy (recommend ≥ 7 days). Manual snapshots before any risky migration (see companion plan's migration-6 hazard, which applies unchanged).
Failover	Synchronous standby in a second Availability Zone; automatic failover on primary failure, typically 60–120 seconds, no application-side connection-string change required if the app connects via the RDS endpoint (not the underlying instance IP).

Digital Integrations

Integration	Description	DR ownership
CGS Associate Authentication API	External identity provider. `identify`, `verifyDomain`, `lookupName` through the AuthProvider interface (CgsAuthProvider in production, AUTH_MODE=cgsauth). No local password store — a successful `identify` is the admission check.	External — owned by the CGS platform team, not this plan.
AWS SES v2 (email)	Outbound notification delivery. EMAIL_MODE=ses in production; templates rendered with React Email. Requires a verified, DKIM-signed sender identity out of the SES sandbox.	Shared — AWS-side availability is AWS's; sender identity, credentials and dispatch scheduling are this team's.
Outbox sweeper	GET /api/internal/dispatch-outbox, called on a schedule (recommended every 5–15 minutes) guarded by DISPATCH_SECRET. The only drain path that runs when nobody is actively booking a trip.	Internal — this team owns the scheduler and the secret.

Servers

Component	Description
Application compute	[TBD — Infrastructure Team to confirm final choice: e.g. AWS ECS/Fargate, Elastic Beanstalk, or EC2 Auto Scaling Group running `pnpm start`]. Recommended to run in at least two Availability Zones behind an Application Load Balancer for the app tier to match the database's Multi-AZ resilience.
Database host	Amazon RDS for PostgreSQL, Multi-AZ (primary + synchronous standby in separate AZs, same VPC/region). Instance class and storage: [TBD — Infrastructure Team to size against production load].

Component	Description
Load balancer	[TBD — AWS Application Load Balancer recommended] in front of the application compute.
Scheduler for the outbox sweeper	Recommended: an Amazon EventBridge scheduled rule invoking the sweeper endpoint every 5–15 minutes with a valid DISPATCH_SECRET header, replacing any host-local cron.

4. Roles and Responsibilities

DR execution follows a small, named chain of ownership rather than being everyone's job. The roles below are organizational roles, not VRS's own application roles (associate / admin_support) — do not confuse the two.

Role	Responsibility during a DR event
DR Plan Owner / IT Manager	Declares the disaster, activates this plan, owns the go/no-go on standing down, and is the single point of authority for the duration of the incident.
Application/Dev Lead	Directs application-tier recovery: redeploys the app, verifies environment configuration, and runs the functional smoke test in §6.
Database Administrator (DBA) / Infra-Cloud Engineer	Directs database recovery: restore, failover, or PITR as appropriate to the incident; verifies schema and data integrity before handing back to the App Lead.
Security / Compliance Officer	Assesses whether the incident involves a security or data-privacy dimension (e.g. suspected breach) and triggers any required regulatory notification track in parallel with technical recovery.
Communications Lead	Owns status updates to stakeholders and end users per §8, so technical responders are not interrupted to answer “any update?”.
Site Admin Coordinators (Manila, Iloilo)	Confirm, once the app is back, that site-specific data (fleet roster, pending approvals) looks correct from the admin side, and relay status to their site's requestors. Mirrors the existing site-based admin notification routing (admin_whitelist.site / notify).

4.1 Contacts

This table is a live document — confirm it is current at the start of every DR test and after every personnel change. Names, emails and phone numbers below are placeholders pending confirmation by the IT Manager; do not treat blank cells as “no one owns this.”

Role	Name	Email	Phone	Escalation order
DR Plan Owner / IT Manager	[TBD]	[TBD]	[TBD]	1
Application/Dev Lead	[TBD]	[TBD]	[TBD]	2

Role	Name	Email	Phone	Escalation order
Database Administrator	[TBD]	[TBD]	[TBD]	2
Infrastructure/Cloud Engineer	[TBD]	[TBD]	[TBD]	3
Security/Compliance Officer	[TBD]	[TBD]	[TBD]	3
Communications Lead	[TBD]	[TBD]	[TBD]	3
Site Admin Coordinator — Manila	[TBD]	[TBD]	[TBD]	4
Site Admin Coordinator — Iloilo	[TBD]	[TBD]	[TBD]	4
CGS Auth Platform Team (external)	[TBD]	[TBD]	[TBD]	on integration failure only

5. Activation Criteria

This plan is activated by the DR Plan Owner when one or more of the following is true. Activation is a formal act — it starts the clock on the RTO and triggers the notification chain in §4.1.

- The application is fully unreachable, or every request 500s, for longer than 15 consecutive minutes.
- The RDS primary instance is unreachable and automatic Multi-AZ failover has not completed within its expected window (~2 minutes), or has completed but the standby is also degraded.
- An AWS regional-level event affects the region hosting RDS/SES/application compute, per the AWS Health Dashboard.
- A failed deployment or migration has broken production and cannot be rolled back within 30 minutes (see the migration-6 hazard in §6).
- A security incident (ransomware, unauthorized access, data exfiltration, compromised IAM credentials) has compromised the application, database, or AWS account.
- Data corruption is discovered that affects the integrity of reservations, the audit trail (reservation_events/roster_events), or the admin whitelist, requiring a point-in-time restore.

Note. Severity levels: SEV1 — full outage or confirmed data loss/corruption, this plan activates immediately. SEV2 — degraded but partially functional (e.g. email delivery down but booking works), the DR Plan Owner decides whether to activate based on duration and business impact. A SEV2 that is not resolved within 2 hours escalates to SEV1 by default.

6. Actions to Implement

Recovery proceeds in the phases below. Each phase names its owner from §4. Phases 3 onward differ meaningfully between the two database platforms and are written for this plan's platform only.

Phase 0 — Detection & Declaration (Owner: DR Plan Owner)

- 1 Confirm the incident against the Activation Criteria in §5 — do not activate on a single unconfirmed alert.
- 2 Formally declare the disaster and record the declaration time; this is T+0 for RTO tracking.
- 3 Open an incident log (timestamp / action / owner / outcome) — required input to the post-incident report in §8.

Phase 1 — Notification (Owner: DR Plan Owner / Communications Lead)

- 1 Notify all contacts in §4.1 in escalation order.
- 2 Post an initial status to the agreed stakeholder channel [TBD — name the channel/distribution list], including expected next update time.

Phase 2 — Assessment (Owner: App Lead + DBA/Infra jointly)

- 1 Determine the failure domain: application, database, network, or an external integration (CGS Auth, SES).
- 2 For an integration-only failure, confirm the application and database are otherwise healthy before treating this as a full DR event — recovery for CGS Auth outages is out of this plan's control (§9) and mainly a communications problem, not a technical one.
- 3 Decide the recovery path for the database (Phase 3) based on the failure type: infrastructure failure vs. data corruption/accidental loss require different actions.

Phase 3 — Database Recovery (Owner: DBA)

- 1 AZ-level failure: confirm RDS Multi-AZ automatic failover has completed (check the RDS event log for a “failover completed” event, typically within ~2 minutes). No manual restore is normally required — the application reconnects using the same RDS endpoint.
- 2 Data corruption / accidental deletion: initiate a point-in-time restore (PITR) to a new RDS instance targeting the last known-good timestamp, minimizing loss toward the ≤ 5 -minute achievable RPO.
- 3 Region-level event: restore the most recent automated backup or cross-region snapshot (if cross-region backup replication is enabled — confirm this is provisioned before relying on it) into a healthy region, and update the application's connection configuration to the new endpoint.
- 4 For a PITR or cross-region restore, cut the application over to the new RDS endpoint via its connection secret (DATABASE_URL / DB_* parameters), not a hand-edited host string, so the change is auditable and reversible.
- 5 Verify row counts and referential integrity on the core tables: reservations, reservation_passengers, admin_whitelist, drivers, vans.
- 6 Verify reference_counters is intact and its per-year sequence has not reset — a reset risks duplicate VR-YYYY-##### reference numbers on the next booking.
- 7 Verify notification_outbox for rows stuck in `sending`; these self-heal via retry/backoff, but confirm none are orphaned past MAX_ATTEMPTS.
- 8 Do NOT re-run `pnpm db:seed` reflexively — see the same caution as the PostgreSQL plan; it reverts legitimate post-seed /roster edits other than active/deactivate status.
- 9 Hand back to the App Lead once the database accepts connections and passes the checks above.

Phase 4 — Application Recovery (Owner: App Lead)

- 1 Provision or restart the application compute per §3.1.
- 2 Verify required environment variables are present and correct: `SESSION_SECRET` (32+ chars), `AUTH_MODE=cgsauth` with a valid `CGS_AUTH_API_KEY`, `EMAIL_MODE=ses` with `AWS_REGION` and `EMAIL_FROM`, the `DATABASE_URL/DB_*` pointing at the recovered database, and `DISPATCH_SECRET`.
- 3 Deploy the last known-good build: ``pnpm install`, `pnpm build`, `pnpm start`` (or restart the existing container/service if the compute itself was not lost).
- 4 Confirm the app is serving: the login page renders and ``GET /api/auth/session`` responds.

Phase 5 — Integration Verification (Owner: App Lead)

- 1 Confirm CGS Auth reachability with a real sign-in attempt at `/admin/login` and `/login`.
- 2 Confirm SES is actually sending, not just queuing: submit a test booking and confirm the corresponding `notification_outbox` row reaches `status='sent'` with a `provider_message_id` set — `status='sent'` alone can also mean the stub, so also confirm `EMAIL_MODE=ses` is set in the running environment.
- 3 Confirm the outbox sweeper's scheduler has resumed calling `GET /api/internal/dispatch-outbox` with the correct `DISPATCH_SECRET` header, and that it returns 200 with a `{sent, failed, retrying}` summary rather than 404 (misconfigured secret) or 401 (wrong secret).

Phase 6 — Data Integrity Verification (Owner: DBA + App Lead)

- 1 Spot-check total reservation counts against the last known-good figure (from monitoring or the last pre-incident report) to bound any silent data loss.
- 2 Scan `reservation_events` and `roster_events` for chronological gaps around the incident window — both are append-only, so a gap is a strong corruption signal.
- 3 Confirm the unique constraints that guard identity and fleet data are intact: `lower(email)` and `lower(domain_id)` on `users`, `lower(trim(email))` on `admin_whitelist`, and `plate` on `vans`.
- 4 Confirm `admin_whitelist.super_admin` holders match the expected roster — if `db:seed` had to be re-run, re-verify who holds it rather than assuming it matches pre-incident state.

Phase 7 — Functional Smoke Test (Owner: App Lead, with a Site Admin Coordinator)

- 1 Sign in as a requestor and submit a test trip through the booking wizard.
- 2 Sign in as an admin, locate the test trip on `/master-list`, and approve it, assigning a driver and van.
- 3 Confirm the requestor received (or, in stub mode only, would have received) the approval notification, and that the trip appears correctly on `/calendar`.
- 4 Confirm the action appears on `/audit-logs`.
- 5 Cancel the test trip to leave no residue in production data.

Phase 8 — Cutover / Go-Live (Owner: DR Plan Owner)

- 1 If the application's public endpoint (APP_URL / DNS) changed as part of recovery, update it and allow for propagation before declaring full availability.
- 2 Declare the system restored and move to §7, Process for Standing Down.

6.1 Recovery Tasks — Time Budget (within the 4-hour RTO)

Phase	Task	Target duration	Cumulative
0–1	Detection, declaration, notification	30 min	0:30
2	Assessment / failure-domain triage	15 min	0:45
3	Database recovery — Multi-AZ failover (typical) or PITR restore (worst case)	5 min typical / up to 1:00 for PITR	0:50 – 1:45
4	Application redeploy/restart and config verification (incl. endpoint cutover if PITR)	30 min	1:20 – 2:15
5	Integration verification (Auth, SES, sweeper)	20 min	1:40 – 2:35
6–7	Data integrity check and functional smoke test	20 min	2:00 – 2:55

Note. This budget shows comfortable headroom inside the 4-hour target even in the worst-case (PITR) scenario, which is the practical argument for the RDS migration alongside the operational-overhead reduction.

7. Process for Standing Down

Standing down closes the incident. It is a deliberate decision by the DR Plan Owner, not an automatic consequence of the smoke test passing.

- 1 Confirm every check in Phase 7 (Functional Smoke Test) passed.
- 2 Hold a soak period of at least 2 hours with normal monitoring in place and no recurrence of the triggering condition before declaring full recovery.
- 3 Turn off any temporary compensating measures put in place during the incident (e.g. a manual email fallback, a maintenance banner, throttled access).
- 4 Notify all §4.1 contacts and the stakeholder channel that the system is fully restored, including the final RTO/RPO actually achieved.
- 5 Formally close the incident log opened in Phase 0, with an end timestamp.
- 6 Schedule a post-incident review within 5 business days (see §8).
- 7 File this plan's corrections — anything that did not match reality during execution (wrong contact, missing step, wrong duration) gets fixed in this document before the next drill, not left for the next real incident to rediscover.

8. Reporting Requirements

During the incident

- Status updates to the stakeholder channel [TBD — name the channel] every 30 minutes during a SEV1, every 60 minutes during a SEV2.
- Each update states: current phase (per §6), time elapsed against the 4-hour RTO, and the next expected update time.
- The incident log (timestamp / action / owner / outcome) is maintained continuously and is the primary source for the post-incident report.

After the incident

- A Post-Incident Report is produced within 5 business days of standing down, owned by the DR Plan Owner.
- Contents: full timeline, root cause, RTO/RPO targeted vs. actually achieved, any confirmed data loss (with the affected reservation/audit reference numbers if applicable), corrective actions with owners and due dates, and sign-off from the DR Plan Owner and the Application Owner.
- If the incident had a security or data-privacy dimension, the Security/Compliance Officer coordinates any required regulatory notification on its own timeline, in parallel with this report — do not wait for the technical report to start that track.
- Reports are retained per company records-retention policy [TBD — confirm retention period with Compliance].

9. Internal–External Dependencies

Internal

Dependency	Why it matters to recovery
Corporate network / VPN access	Responders need connectivity to the hosting environment and to each other's tools during recovery.
Source control (the VRS git repository)	The application is rebuilt from source on redeploy; the repository's availability is a hard dependency for Phase 4.
Package registry (npm/pnpm registry)	`pnpm install` during redeploy needs registry access; an outage there blocks Phase 4 independently of this system's own infrastructure.
Infrastructure/Ops team	Owns the physical or virtual hosts, network, and (for the PostgreSQL plan) backup tooling this plan assumes exists.

External

Dependency	Why it matters to recovery	Controlled by
CGS Associate Authentication API	No one can sign in — as requestor or admin — while this is down. This plan cannot recover it; recovery is limited to detecting and communicating the outage.	CGS platform team
AWS (RDS, SES, region/AZ health)	The database platform and email delivery both live on AWS; a regional AWS event affects both simultaneously, which is why Phase 3's region-level path exists.	AWS
AWS account access / IAM	Recovery actions (failover confirmation, PITR, snapshot restore) require IAM permissions scoped to the responders in §4; access must survive whatever caused the incident.	Company Infrastructure/Cloud team
DNS provider	Needed if the application's public endpoint must be repointed during recovery.	[TBD — name the provider]

Note. Any dependency this team does not control (CGS Auth, AWS platform health) is explicitly out of this plan's ability to recover — the plan's job for those is detection, communication, and waiting, not remediation.

Appendix A: Application Diagram

Requestor/Admin Browser → Application Load Balancer → Next.js App instances (App Router; edge middleware gates admin pages) → API routes → domain modules (auth, reservations, admins, drivers, vans, roster, audit, email) → Kysely → Amazon RDS for PostgreSQL (Multi-AZ: primary + synchronous standby, automated backups with point-in-time recovery). Two external dependencies sit beside the app: the CGS Associate Authentication API (identity) and AWS SES (outbound email). An EventBridge-scheduled rule calls the internal outbox sweeper endpoint on an interval to drain notification_outbox independently of live traffic.

